International Cybersecurity and Digital Forensics Academy (ICDFA)

CIP-B101: Basic Computer Skills for Digital Forensics

Lab 1A: Number Systems, ASCII, Timestamps and Data Representation

Course Code	CIP-B101
Lab Title	Lab 1A: Number Systems, ASCII, Timestamps and Data Representation
Full Name	Fuseini Imoru Kantuogaa
Student ID	C11/26/DFIT/17291
Date	25th June 2026
Platform Used	Kali Linux

Task A - Number Systems and Manual Conversion

2.2 Practical Conversion Exercises

1. Convert 101010_2 to decimal (weight of each bit from right to left):

Position: 5 4 3 2 1 0

Bit: 1 0 1 0 1 0

Weight: 32 0 8 0 2 0

Result: 32 + 0 + 8 + 0 + 2 + 0 = 42

2. Convert $AB2_{16}$ to decimal (A = 10, B = 11):

$A \times 16^2 + B \times 16^1 + 2 \times 16^0$

$= 10 \times 256 + 11 \times 16 + 2 \times 1$

$= 2560 + 176 + 2 = 2738$

3. Convert 126_{10} to binary (division by 2):

126 ÷ 2 = 63 remainder 0

63 ÷ 2 = 31 remainder 1

31 ÷ 2 = 15 remainder 1

15 ÷ 2 = 7 remainder 1

7 ÷ 2 = 3 remainder 1

3 ÷ 2 = 1 remainder 1

1 ÷ 2 = 0 remainder 1

Read remainders bottom to top: 1111110

4. Convert 2738_{10} to hexadecimal (division by 16):

2738 ÷ 16 = 171 remainder 2

171 ÷ 16 = 10 remainder 11 (B)

10 = A

Read remainders bottom to top: AB2

5. Convert 101010110010_2 to hexadecimal (group bits in fours from the right):

1010 | 1011 | 0010

A | B | 2 = AB2

2.3 Validate With Linux Shell and Python (Screenshots)

Screenshot 1: Working folder creation and shell validation commands (echo $((2#101010)) = 42, echo $((16#AB2)) = 2738)

Screenshot 2: bc conversion (obase=2;126 = 1111110) and Python validation (int('101010',2)=42, int('AB2',16)=2738, bin(126)=0b1111110, hex(2738)=0xab2)

2.4 Evidence Table

Evidence Value	Given Base	Manual Answer	Command Used to Verify	Screenshot Ref.
101010	Binary	42 (decimal)	echo $((2#101010))	Screenshot 1
AB2	Hexadecimal	2738 (decimal)	echo $((16#AB2))	Screenshot 1
126	Decimal	1111110 (binary)	echo "obase=2;126" | bc	Screenshot 2
2738	Decimal	AB2 (hexadecimal)	printf "%X" 2738	Screenshot 2
101010110010	Binary	AB2 (hexadecimal)	python3: hex(int('101010110010',2))	Screenshot 2

Task B - ASCII, UTF-8 and Hex Evidence Strings

3.1 Create Text Evidence and View as Hex

Command: echo -n "Hello" > evidence_text.txt

xxd output: 00000000: 4865 6c6c 6f Hello

Plain hex (xxd -p): 48656c6c6f

3.2 Convert Hex to ASCII

printf "48656c6c6f" | xxd -r -p → Hello

3.3 Python Conversion (Screenshot 4)

ASCII bytes: b'Hello'

ASCII to hex: 48656c6c6f

Hex to ASCII: Hello

ord('H'): 72

chr(72): H

3.4 Student Exercise

File created: Fuseini_lab1a.txt containing: Digital Forensics Begins with Bytes.

xxd hex dump (Screenshot 5):

00000000: 4469 6769 7461 6c20 466f 7265 6e73 6963 Digital Forensic

00000010: 7320 4265 6769 6e73 2057 6974 6820 4279 s Begins With By

00000020: 7465 732e 0a   tes..

Plain hex (xxd -p): 4469676974616c20466f72656e736963732042656769...

Hex converted back to text: Digital Forensics Begins With Bytes.

Why does 0A appear at the end when echo is used without -n?

When echo is used (without -n), it automatically appends a newline character (0x0A) at the end of the output. This is the standard Unix/Linux line feed character. The -n flag suppresses this newline, so the file contains only the exact characters typed. In digital forensics, this distinction is critical because even a single extra byte like 0x0A changes the file hash entirely.

Forensic Explanation; Not All Hex is Readable Text:

An examiner should not assume every hexadecimal value represents readable ASCII text. Bytes can represent binary structures such as file headers (e.g., the MZ header 4D 5A for Windows executables), compressed data, encrypted content, image pixel data, or raw numeric values. Blindly converting hex to ASCII can produce garbled output or misleading interpretations. A forensic analyst must understand the context of the data source before attempting any interpretation.

Task C - Epoch Time and Forensic Timestamps

4.1 Unix Epoch Conversion

Current Unix timestamp (date +%s): 1782253076 (captured at time of lab execution, June 2026)

1643643491 converted: Mon Jan 31 03:38:11 PM UTC 2022 (date -u -d @1643643491)

Python confirmation: 2022-01-31 15:38:11+00:00

4.2 Compare Local Time and UTC (Screenshot 7)

Local time: Tue Jun 23 06:19:24 PM EDT 2026

UTC time: Tue Jun 23 10:19:24 PM UTC 2026

Python: Local: 2026-06-23 18:19:24.956634 | UTC: 2026-06-23 22:19:24.956854+00:00

4.3 Mac Absolute Time / Cocoa Core Date (Screenshot 8)

Mac epoch offset from Unix epoch: 978307200 seconds

Mac Absolute Time value: 725846400

Unix equivalent: 725846400 + 978307200 = 1704153600

Readable UTC result: 2024-01-02 00:00:00+00:00

4.4 Timestamp Worksheet

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot
1643643491	Unix seconds	date -u -d @1643643491	2022-01-31 15:38:11 UTC	Screenshot 6
1675008832	Unix seconds	python3: datetime.fromtimestamp(1675008832, tz=timezone.utc)	2023-01-29 16:13:52 UTC	Verified
725846400	Mac Absolute Time	unix = 725846400 + 978307200; datetime.fromtimestamp(unix, utc)	2024-01-02 00:00:00 UTC	Screenshot 8
Current time	Unix seconds	date +%s	1782253076 (June 2026)	Screenshot 6

Important Timeline Warning:

All timestamp results in this report are stated in UTC. Failing to distinguish UTC from local time is a serious error in forensic analysis the same Unix timestamp can appear hours earlier or later depending on the system time zone. All forensic timelines must explicitly state the time zone of every reported timestamp.

Task D - Hashing and the Newline Problem

5.1 File Creation and xxd Comparison (Screenshots 8 & 9)

File 1: hello_with_newline.txt – created with echo "Hello" → contains bytes: 48 65 6c 6c 6f 0a

File 2: hello_without_newline.txt – created with echo -n "Hello" → contains bytes: 48 65 6c 6c 6f

The visible content appears identical but the byte sequences differ by the trailing 0x0A newline byte.

Hash Results

File	MD5 Hash	SHA-256 Hash
hello_with_newline.txt	09f7e02f1290be211da707a266f153b3	66a045b452102c59d840ec097d59d9467e13a3f34f6494e539ffd32c1bb35f18
hello_without_newline.txt	8b1a9953c4611296a827abf8c47804d7	185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

Explanation

The two files produce completely different MD5 and SHA-256 hash values even though their visible text looks identical. This is because cryptographic hash functions operate on raw bytes, not on what a human would read. File 1 has 6 bytes (including the 0x0A newline); File 2 has only 5 bytes. Even a one-byte difference causes an entirely different hash output. This demonstrates the byte-sensitivity of cryptographic hashing.

To hash only the visible characters (without newline), echo -n should be used. This ensures the hash reflects exactly the text content and nothing more.

Task E - Endianness and Data Stored in Memory

6.1 System Byte Order Check (Screenshot 9)

System byte order: little

This means the Kali Linux system stores multi-byte values with the least significant byte first.

6.2 Endianness Exercise Worksheet

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
01000000	1	16777216	int.from_bytes(bytes.fromhex('01000000'), 'little'/'big')	LE reads 0x01 as least significant; BE reads it as most significant
FF000000	255	4278190080	int.from_bytes(bytes.fromhex('FF000000'), 'little'/'big')	LE: 0xFF = 255; BE: 0xFF000000 = 4278190080
00000100	65536	256	int.from_bytes(bytes.fromhex('00000100'), 'little'/'big')	LE: 0x00010000; BE: 0x00000100 = 256
78563412	305419896	2018915346	int.from_bytes(bytes.fromhex('78563412'), 'little'/'big')	LE: 0x12345678; BE: 0x78563412

Forensic Explanation – Same Bytes, Different Values:

The same four bytes can produce entirely different decimal values depending on byte order. In little-endian systems (such as most modern Intel/AMD processors), the least significant byte is stored first in memory. So, the bytes 78 56 34 12 are read as 0x12345678 = 305,419,896. In big-endian systems, the most significant byte comes first, so the same bytes are read as 0x78563412 = 2,018,915,346. When analysing disk metadata, FAT file systems and NTFS structures store values in little-endian order. Misinterpreting byte order when reading a file size or timestamp from a sector dump would produce completely wrong evidence values. A forensic analyst must always know the byte order of the system being examined before interpreting multi-byte fields.

Task F - Mini Forensic Interpretation Case

7.0 Evidence Interpretation Table

Evidence Item	**Value**	**Student Interpretation**	**Command / Method**
EVID-001	48656c6c6f204943444641	ASCII text: "Hello ICDFA"	binascii.unhexlify('48656c6c6f20494344641').decode()
EVID-002	01001000 01101001	Binary-encoded ASCII: "Hi" (01001000 = H = 72; 01101001 = i = 105)	chr(int('01001000',2)) + chr(int('01101001',2))
EVID-003	1675008832	Unix timestamp: 2023-01-29 16:13:52 UTC	datetime.fromtimestamp(1675008832, tz=timezone.utc)
EVID-004	725846400 (Mac Absolute Time)	Mac Absolute Time: add 978307200 offset → 2024-01-02 00:00:00 UTC	datetime.fromtimestamp(725846400 + 978307200, tz=timezone.utc)
EVID-005	78563412 (little-endian and big-endian)	Little-endian: 305,419,896 (0x12345678); Big-endian:	int.from_bytes(bytes.fromhex('78563412'), 'little'/'big')

		2,018,915,346 (0x78563412)	

7.1 Case Narrative

EVID-001 – Text/Content Interpretation:

The hex string 48656c6c6f204943444641 decodes to the ASCII text "Hello ICDFA". This is useful for content interpretation – it reveals a human-readable message that could identify a file, text string, or data payload recovered from a disk or network capture.

EVID-002 – Binary-Encoded Text:

The two binary values 01001000 and 01101001 correspond to decimal values 72 and 105 respectively, which are the ASCII codes for the characters 'H' and 'i'. Together they spell "Hi". This demonstrates that binary representations at the byte level can encode text just as hexadecimal can.

EVID-003 – Timeline Evidence (Unix Timestamp):

The Unix timestamp 1675008832 converts to 2023-01-29 16:13:52 UTC. This is the most directly useful value for timeline analysis, as it gives an unambiguous point in time that can be placed on a forensic timeline. It could represent a file creation time, log entry, or user action.

EVID-004 – Timeline Evidence (Mac Absolute Time):

The Mac Absolute Time value 725846400 requires adding the offset of 978307200 seconds (the difference between the Unix epoch of 1970-01-01 and the Mac epoch of 2001-01-01) to obtain the Unix equivalent of 1704153600, which converts to 2024-01-02 00:00:00 UTC. This is also useful for timeline analysis in macOS artefact investigations.

EVID-005 – Byte Order Evidence:

The four bytes 78 56 34 12 produce two different values depending on byte order. In little-endian, they represent 0x12345678 = 305,419,896. In big-endian, they represent 0x78563412 = 2,018,915,346. Without knowing the byte order of the source system, these bytes could be misinterpreted, potentially placing a wrong file size, offset, or timestamp in a forensic report. This evidence item is most relevant to interpreting disk structures and memory dumps.

Minimum Screenshot Checklist

No.	Screenshot Evidence	Included?
1	Working folder creation and pwd output	Yes
2	Binary/hex/decimal conversion commands	Yes

3	xxd output for text evidence	Yes
4	Hex-to-ASCII conversion output	Yes
5	Unix timestamp conversion output	Yes
6	Mac Absolute Time conversion output	Yes
7	Hash comparison using md5sum and sha256sum	Yes
8	Endianness Python output	Yes
9	Mini-case evidence interpretation commands	Yes

Reflection

Number systems and timestamps are fundamental building blocks of digital forensics. Every piece of data stored on a computer whether it is a file, a log entry, a network packet, or a disk sector ultimately exists as a sequence of bytes that can be represented in binary, decimal, or hexadecimal. Without the ability to move fluently between these representations, a forensic examiner cannot read raw disk images, interpret file headers, or understand memory dumps.

Timestamps, in particular, are central to the construction of forensic timelines. An error in timestamp conversion such as confusing UTC with a local time zone, or failing to recognise that a value uses the Mac Absolute Time epoch rather than the Unix epoch can place a suspect action at the wrong time, potentially invalidating an entire investigation. The exercises in this lab demonstrated that the same numeric value means something entirely different depending on the epoch system used.

The hashing exercise reinforced that even a single invisible byte (the newline character 0x0A) is enough to produce a completely different hash value, which highlights why hash verification must be performed with precision. The endianness exercises demonstrated that raw bytes have no inherent meaning without knowing the architecture of the source system.

In summary, number systems and timestamps are not abstract mathematical concepts they are practical tools that determine whether digital evidence is correctly identified, accurately placed on a timeline, and reliably verified. Mastering these fundamentals is essential before advancing to higher-level forensic analysis.